
LAPORAN PRAKTIKUM

IOT 2

Module 4 & Module 5



Oleh :

Nama : Muhammad Raid Sadad Kussyamsuddin

NPM : 25782076

Kelas : TRI (3C)

Dosen Pengampu : Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

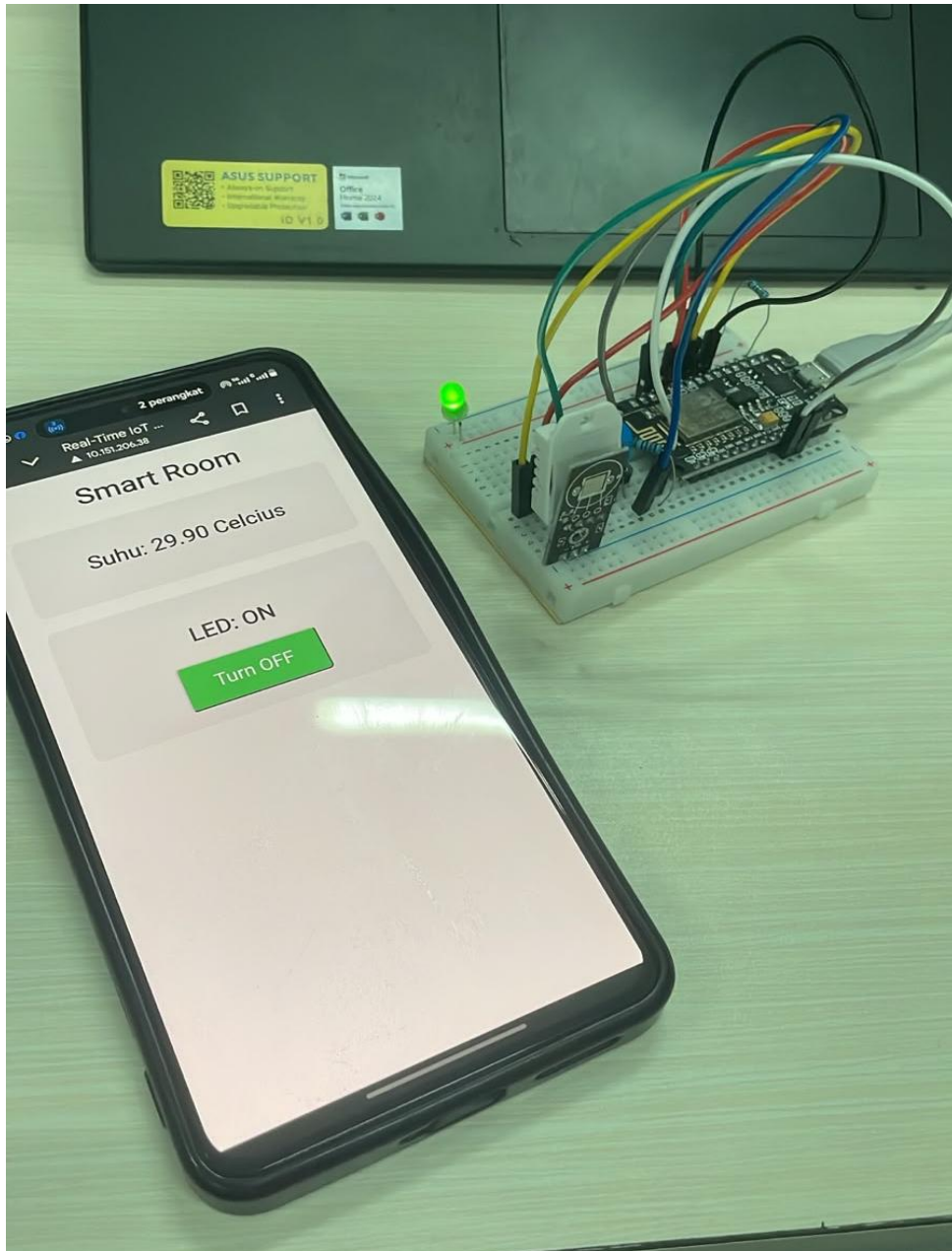
POLITEKNIK NEGERI LAMPUNG

TAHUN 2026

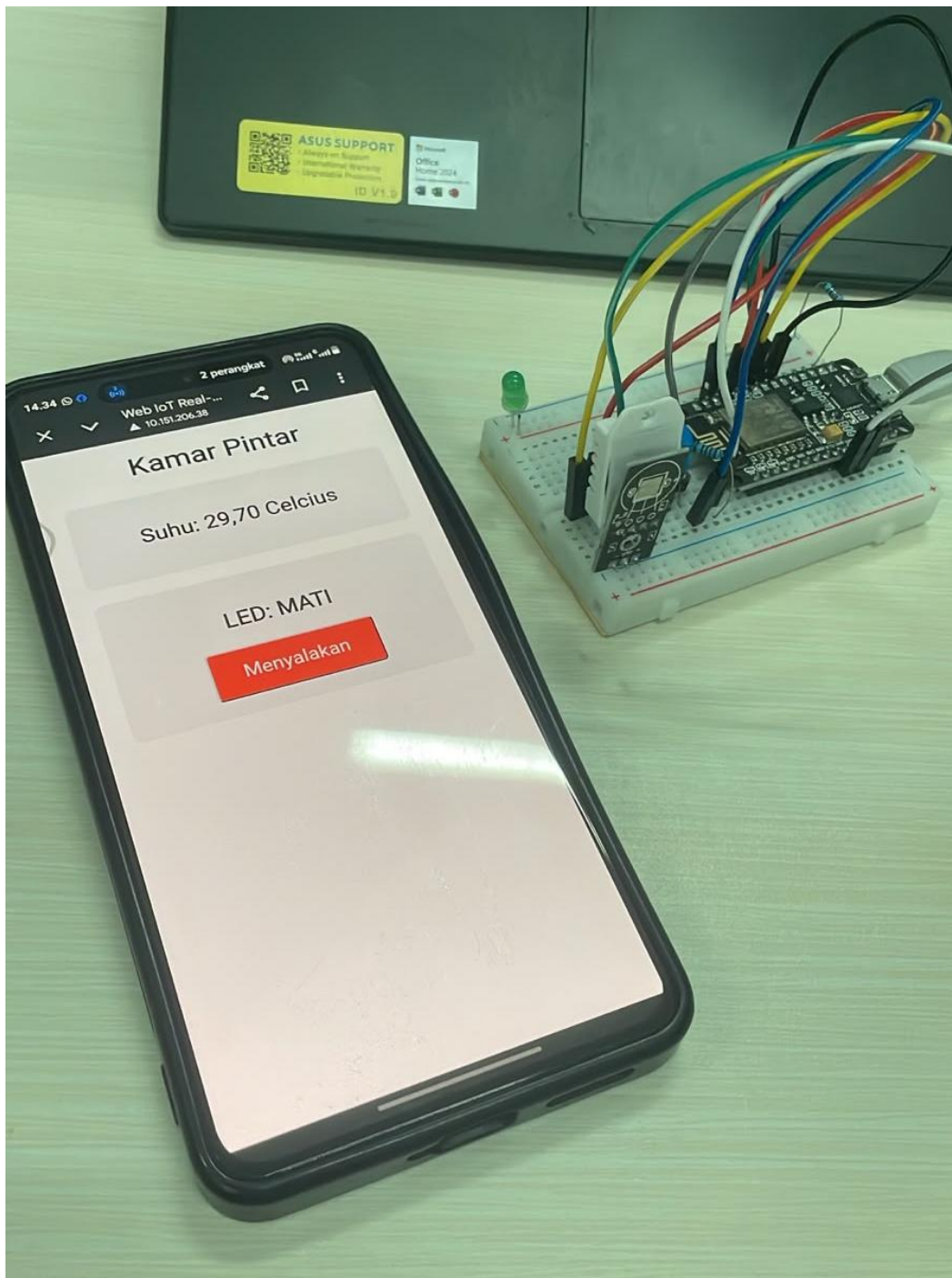
4.5.1 Praktikum 1: *Real-Time Dashboard* & Sinkronisasi Tombol Fisik

Nah disini saya sudah berhasil melaksanakan praktikum 1 pada module 4 yang dimana lampu dapat dimatikan dan dihidupkan melalui handphone atau web dan disini di bagian kode program kita menggunakan web socket agar data atau informasi yang ditangkap oleh sensor dapat dikirim ke web secara real time, tidak perlu me refresh halaman web

Gambar LED ON :



Gambar LED OFF :



4.6 Latihan

1. Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Jawaban : Karena websocket itu membuat koneksi tetap terbuka antara ESP8266 dan browser, sedangkan kalo Meta Refresh harus melakukan refresh pada halaman web yang dimana lebih banyak menggunakan bandwidth dan dapat menyebabkan layar berkedip

2. Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

Jawaban : dampak jika menggunakan `delay(300)` pada `loop()` akan menghentikan proses ESP8266 selama 3 detik, dampaknya websocket dan tombol menjadi kurang responsive karena ESP8266 tidak dapat memproses perintah lainnya selama delay ini berjalan. Karena itu pada program digunakan `milis()` agar pembacaan sensor dapat dilakukan tanpa menghentikan proses lainnya.

3. Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

Jawaban : Fungsi dari `onMessage()` adalah menerima data dari websocket dalam bentuk JSON, kemudian mengambil data tersebut lalu memperbarui tampilan pada halaman web. Pada elemen HTML yang di perbarui adalah `tempValue` yaitu menampilkan suhu, `ledStatus` yaitu menampilkan status LED, dan `toggleBtn` yaitu mengubah tulisan tombol menjadi Turn ON atau Turn OFF

4. Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via *WebSockets*:

- i. Sisi Back-End (C++): Buat variabel penampung (misal: `String currentHum = "--";`). Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer *non-blocking* `millis()`. Perbarui fungsi `notifyClients()` agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: `{"led":"...", "suhu":"...", "hum":"..."}`).
- ii. Sisi Front-End (HTML & JavaScript): Tambahkan elemen kartu (`<div class="card">`) baru pada kerangka HTML sebagai penampil kelembapan

(misal: <h2>Kelembapan: -- %</h2>).
Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue.
Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

Jawaban : Saya sudah berhasil mencoba latihan no 4 yang dimana berhasil menambahkan monitoring kelembapan

Langkah awal saya menambahkan variabel kelembapan dengan perintah

```
DHT dht(dhtPin, DHT22);

// Variabel Pelacak Status (State & Cache)
bool ledState = false;
String currentTemp = "--";
String currentHum = "--";
int buttonState;
int lastButtonState = LOW;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;
unsigned long lastTime = 0;
```

Langkah yang kedua saya menambahkan baris kode dibawah untuk menambahkan kelembapan ke data json :

```
void notifyClients() {
  String jsonString = "{\"led\": \"\" + String(ledState ? 1 : 0) + "\", ";
  jsonString += "\"suhu\": \"\" + currentTemp + "\", ";
  jsonString += "\"kelembapan\": \"\" + currentHum + "\"}";
  ws.textAll(jsonString);
}
```

Langkah yang ketiga saya menambahkan baris kode dibawah untuk menambahkan tampilan kelembapan ke HTML :

```
</head>
<body>
  <h1>Smart Room</h1>
  <div class="card">
    <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
  </div>
  <div class="card">
    <h2>Kelembapan: <span id="humValue">--</span> %</h2>
  </div>
  <div class="card">
    <h2>LED: <span id="ledStatus">OFF</span></h2>
    <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn ON</button>
  </div>
```

Langkah keempat saya menambahkan baris kode untuk membaca kelembapan dengan baris perintah :

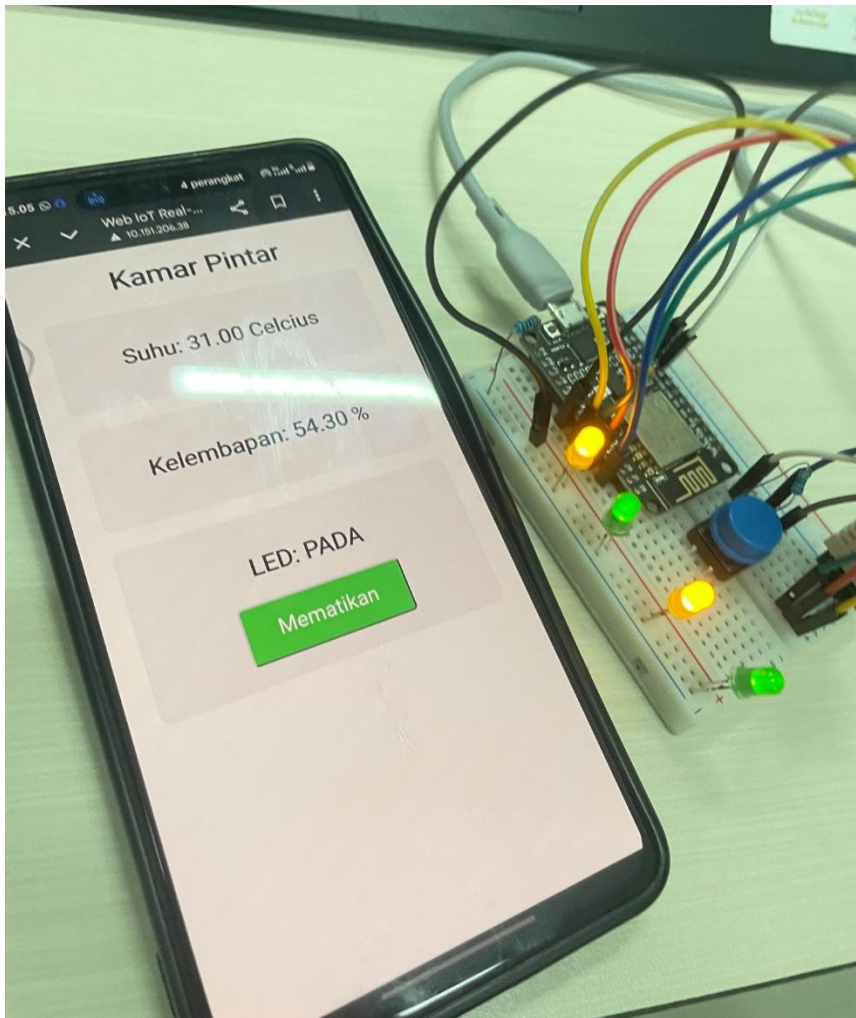
```
// 3. Baca DHT setiap 3 detik di memori cache, tanpa memblokir CPU
if ((millis() - lastTime) > 3000) {
    float t = dht.readTemperature();
    float h = dht.readHumidity();
    if(!isnan(t)) {
        currentTemp = String(t);
    }
    if(!isnan(h)) {
        currentHum = String(h);
    }
    notifyClients();
    lastTime = millis();
}
```

Langkah kelima saya menambahkan baris kode dibawah untuk menampilkan data kelembapan yang di terima dari esp8266 secara real time pada halaman web

```
// Menyuntikkan teks angka Suhu ke dalam id "tempValue" HTML
if(dataObj.suhu !== undefined) {
    document.getElementById('tempValue').innerHTML = dataObj.suhu;
}

if(dataObj.kelembapan !== undefined) {
    document.getElementById('humValue').innerHTML = dataObj.kelembapan;
}
```

Foto bukti latihan no 4 :

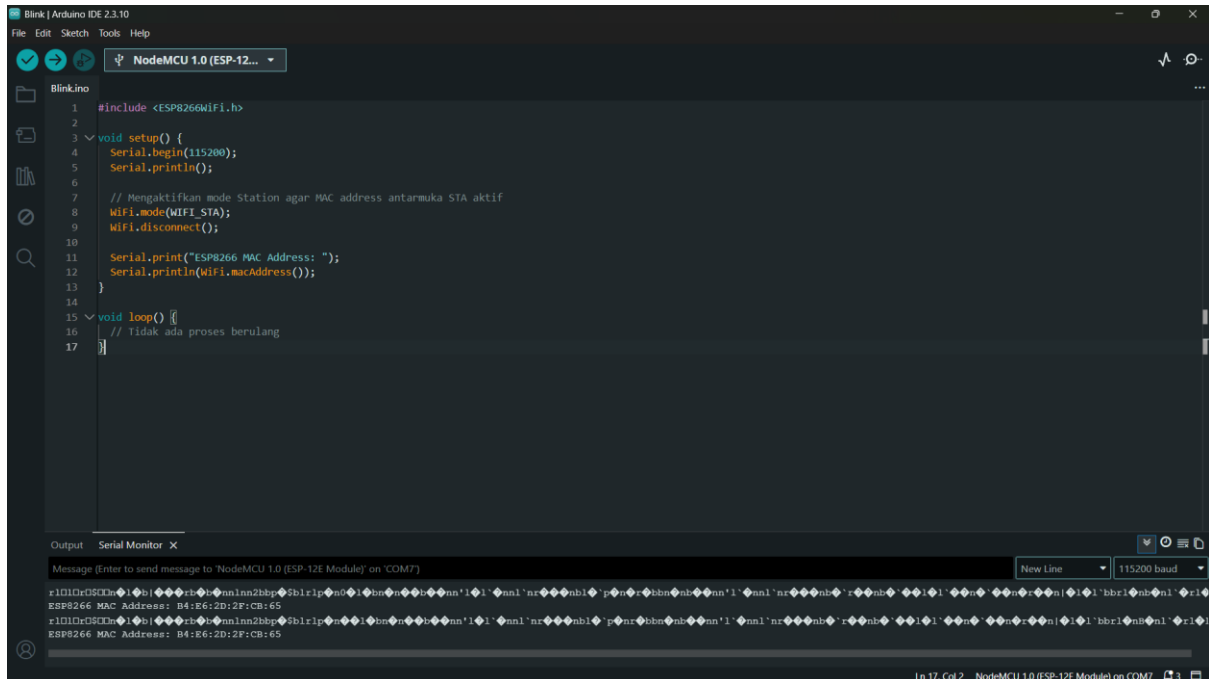


Bukti Terhubung :

```
Praktikum_1 | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12E...)
Praktikum_1.ino
1 #include <ESP8266WiFi.h>
2 #include <ESPAsyncTCP.h>
3 #include <ESPAsyncWebServer.h>
4 #include <DHT.h>
5
6 const char* ssid = "V2030";
7 const char* password = "raid_0507";
8
9 // Konfigurasi Pin
10 const byte dhtPin = 2; // D4 (GPIO 2)
11 const byte buttonPin = 4; // D2 (GPIO 4)
12 const byte ledPin = 12; // D6 (GPIO 12)
13
14 DHT dht(dhtPin, DHT22);
15
16 // Variabel Pelacak Status (State & Cache)
17 bool ledState = false;
18 String currentTemp = "...";
19 int buttonState;
20 int lastButtonState = LOW;
21 unsigned long lastDebounceTime = 0;
22 const unsigned long debounceDelay = 50;
23 unsigned long lastTime = 0;
24
25 void setup() {
26   pinMode(dhtPin, INPUT);
27   pinMode(buttonPin, INPUT_PULLUP);
28   pinMode(ledPin, OUTPUT);
29   WiFi.begin(ssid, password);
30   while (WiFi.status() != WL_CONNECTED) {
31     delay(500);
32     Serial.print(".");
33   }
34   Serial.println("WiFi connected");
35   Serial.print("IP Address: ");
36   Serial.println(WiFi.localIP());
37   dht.begin();
38 }
39
40 void loop() {
41   // Read the temperature and humidity
42   float temp = dht.temperature();
43   float hum = dht.humidity();
44   String tempStr = String(temp, 2);
45   String humStr = String(hum, 2);
46
47   // Read the button state
48   int buttonState = digitalRead(buttonPin);
49
50   // Check if the button has been pressed
51   if (buttonState == LOW && lastButtonState == HIGH) {
52     // Toggle the LED state
53     ledState = !ledState;
54     digitalWrite(ledPin, ledState);
55     lastButtonState = buttonState;
56     lastTime = millis();
57   }
58
59   // Send the temperature and humidity data to the web server
60   AsyncWebServer *server = new AsyncWebServer(80);
61   server->on("/temp", HTTP_GET, []() {
62     AsyncWebResponse *res = new AsyncWebResponse(200, "text/plain");
63     res->setContent(tempStr);
64     server->send(res);
65   });
66   server->on("/hum", HTTP_GET, []() {
67     AsyncWebResponse *res = new AsyncWebResponse(200, "text/plain");
68     res->setContent(humStr);
69     server->send(res);
70   });
71   server->begin();
72 }
73
74 Serial Monitor X Output
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')
Client Websocket #7 terhubung
IP Address: 10.151.206.38
Ln 1, Col 1 NodeMCU 1.0 (ESP-12E Module) on COM7
```

Modul 5: Topologi Jaringan Terdesentralisasi (ESP-NOW dan Mesh)

5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU



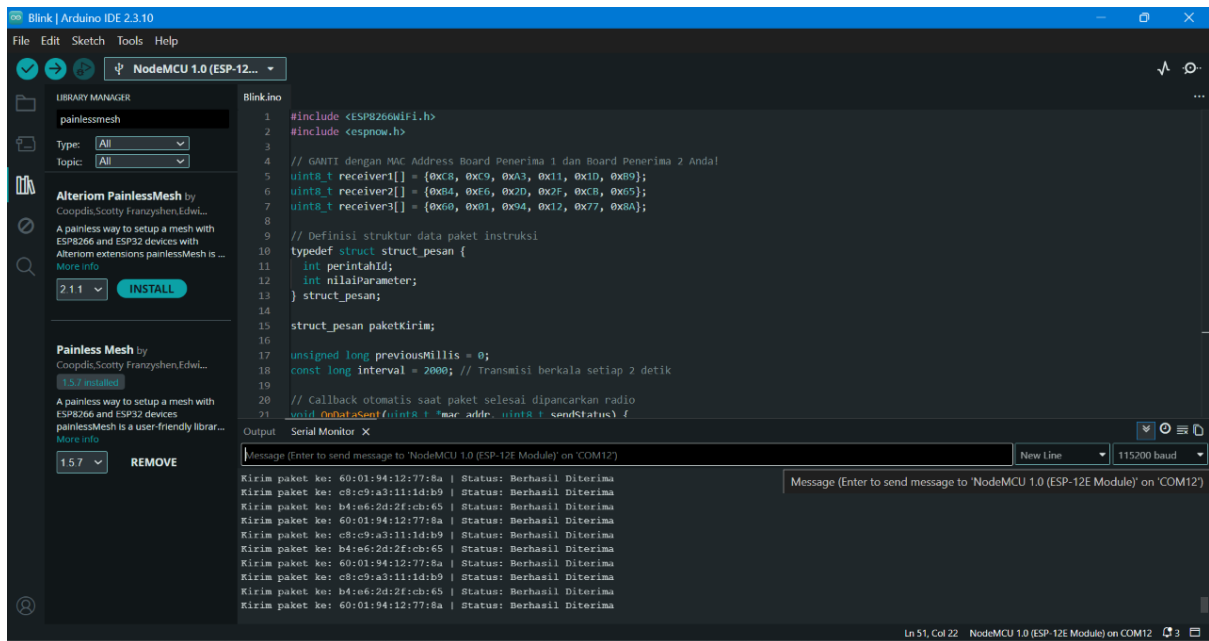
The screenshot shows the Arduino IDE interface with a sketch named 'Blink.ino'. The code is as follows:

```
1 #include <ESP8266WiFi.h>
2
3 void setup() {
4   Serial.begin(115200);
5   Serial.println();
6
7   // Mengaktifkan mode Station agar MAC address antarmuka STA aktif
8   WiFi.mode(WIFI_STA);
9   WiFi.disconnect();
10
11   Serial.print("ESP8266 MAC Address: ");
12   Serial.println(WiFi.macAddress());
13 }
14
15 void loop() {
16   // Tidak ada proses berulang
17 }
```

The Serial Monitor at the bottom shows the output: 'ESP8266 MAC Address: B4:E6:2D:2F:CB:65'.

5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

Controller

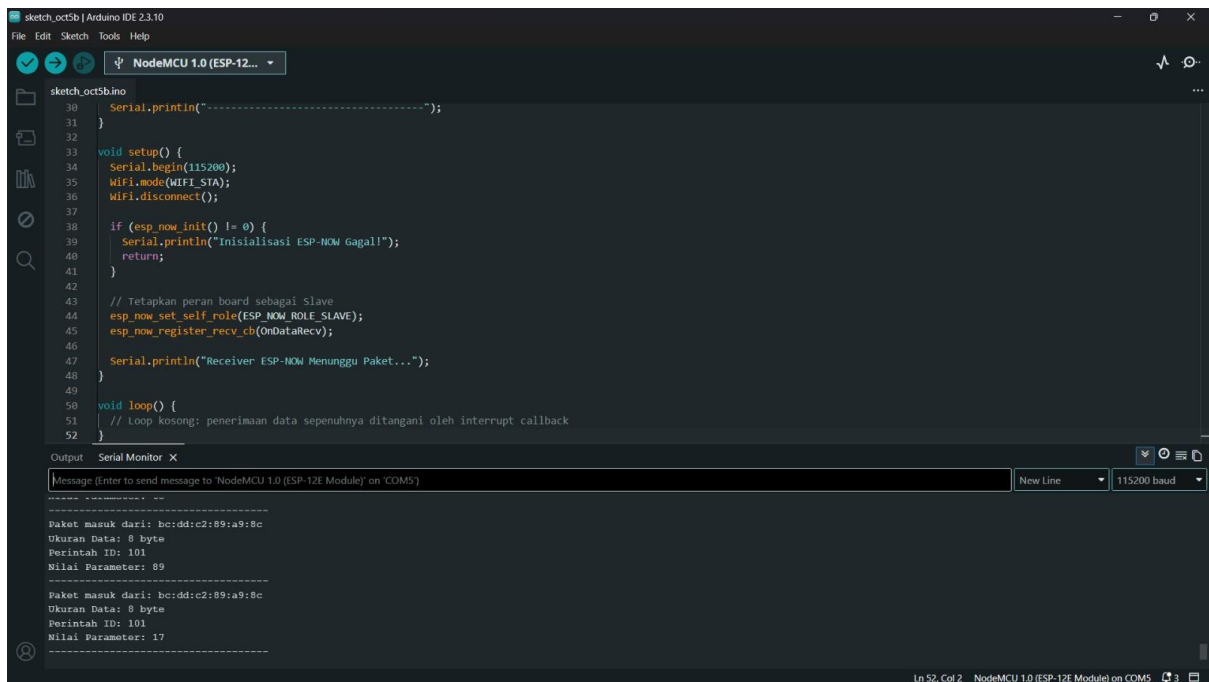


The screenshot shows the Arduino IDE interface with a sketch named 'Blink.ino'. The code is as follows:

```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Andai
5 uint8_t receiver1[] = {0xC8, 0xC9, 0xA3, 0x11, 0xD, 0xB9};
6 uint8_t receiver2[] = {0xC8, 0xF6, 0x2D, 0x2F, 0xCB, 0x65};
7 uint8_t receiver3[] = {0xC8, 0xA3, 0x11, 0xD, 0xB9};
8
9 // Definisi struktur data paket instruksi
10 typedef struct struct_pesan {
11   int perintahId;
12   int nilaiParameter;
13 } struct_pesan;
14
15 struct_pesan paketKirim;
16
17 unsigned long previousMillis = 0;
18 const long interval = 2000; // Transmisi berkala setiap 2 detik
19
20 // Callback otomatis saat paket selesai dipancarkan radio
21 void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
```

The Serial Monitor at the bottom shows the output: 'Kirim paket ke: 60:01:94:12:77:8a | Status: Berhasil Diterima'.

Penerima 1



```
sketch_oct5bino
30   Serial.println("-----");
31 }
32
33 void setup() {
34   Serial.begin(115200);
35   WiFi.mode(WIFI_STA);
36   WiFi.disconnect();
37
38   if (esp_now_init() != 0) {
39     Serial.println("Inisialisasi ESP-NOW gagal!");
40     return;
41   }
42
43   // Tetapkan peran board sebagai Slave
44   esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
45   esp_now_register_recv_cb(OnDataRecv);
46
47   Serial.println("Receiver: ESP-NOW Menunggu Paket...");
48 }
49
50 void loop() {
51   // Loop kosong: penerimaan data sepenuhnya ditangani oleh interrupt callback
52 }
```

Output Serial Monitor X

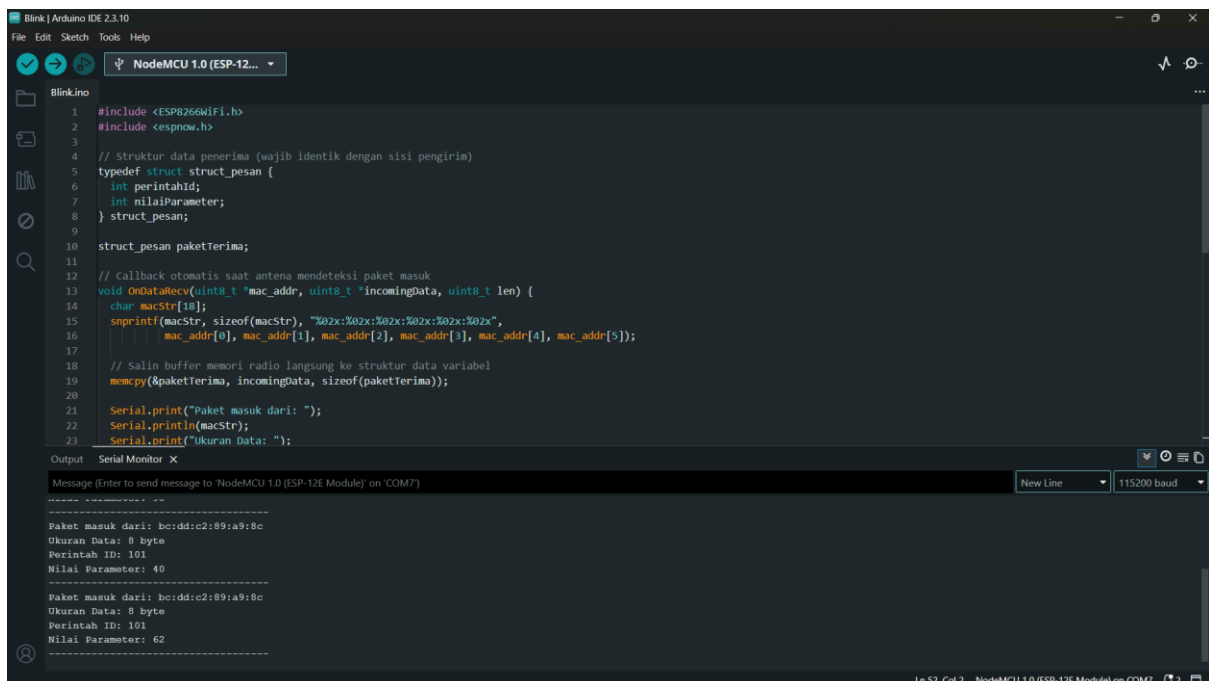
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

Paket masuk dari: bc:dd:c2:89:a9:8c
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 89

Paket masuk dari: bc:dd:c2:89:a9:8c
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 17

Ln 52, Col 2 NodeMCU 1.0 (ESP-12E Module) on COM5

Penerima 2



```
Blinkino
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // Struktur data penerima (wajib identik dengan sisi pengirim)
5 typedef struct struct_pesan {
6   int perintahid;
7   int nilaiParameter;
8 } struct_pesan;
9
10 struct_pesan paketTerima;
11
12 // Callback otomatis saat antenna mendeteksi paket masuk
13 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14   char macStr[18];
15   snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
16            mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17
18   // Salin buffer memori radio langsung ke struktur data variabel
19   memcpy(&paketTerima, incomingData, sizeof(paketTerima));
20
21   Serial.print("Paket masuk dari: ");
22   Serial.println(macStr);
23   Serial.print("Ukuran Data: ");
```

Output Serial Monitor X

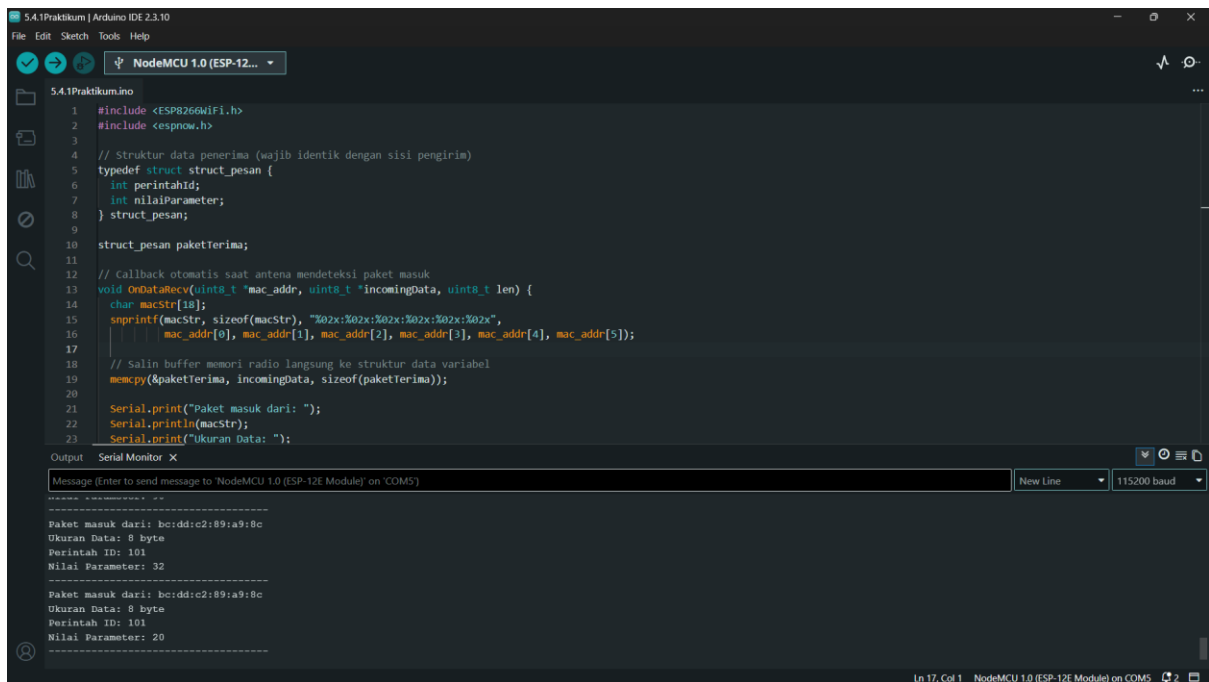
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')

Paket masuk dari: bc:dd:c2:89:a9:8c
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 40

Paket masuk dari: bc:dd:c2:89:a9:8c
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 62

Ln 52, Col 2 NodeMCU 1.0 (ESP-12E Module) on COM7

Penerima 3



The screenshot shows the Arduino IDE interface with a sketch named "5.4.1Praktikum3.ino" for a NodeMCU 1.0 (ESP-12E) module. The code defines a structure for incoming data and a callback function to process it. The serial monitor shows two received packets with their respective data and parameters.

```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // Struktur data penerima (wajib identik dengan sisi pengirim)
5 typedef struct struct_pesan {
6   int perintahId;
7   int nilaiParameter;
8 } struct_pesan;
9
10 struct_pesan paketTerima;
11
12 // Callback otomatis saat antenna mendeteksi paket masuk
13 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14   char macStr[18];
15   sprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x",
16           mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17
18   // Salin buffer memori radio langsung ke struktur data variabel
19   memcpy(&paketTerima, incomingData, sizeof(paketTerima));
20
21   Serial.print("Paket masuk dari: ");
22   Serial.println(macStr);
23   Serial.print("Ukuran Data: ");
```

Serial Monitor X

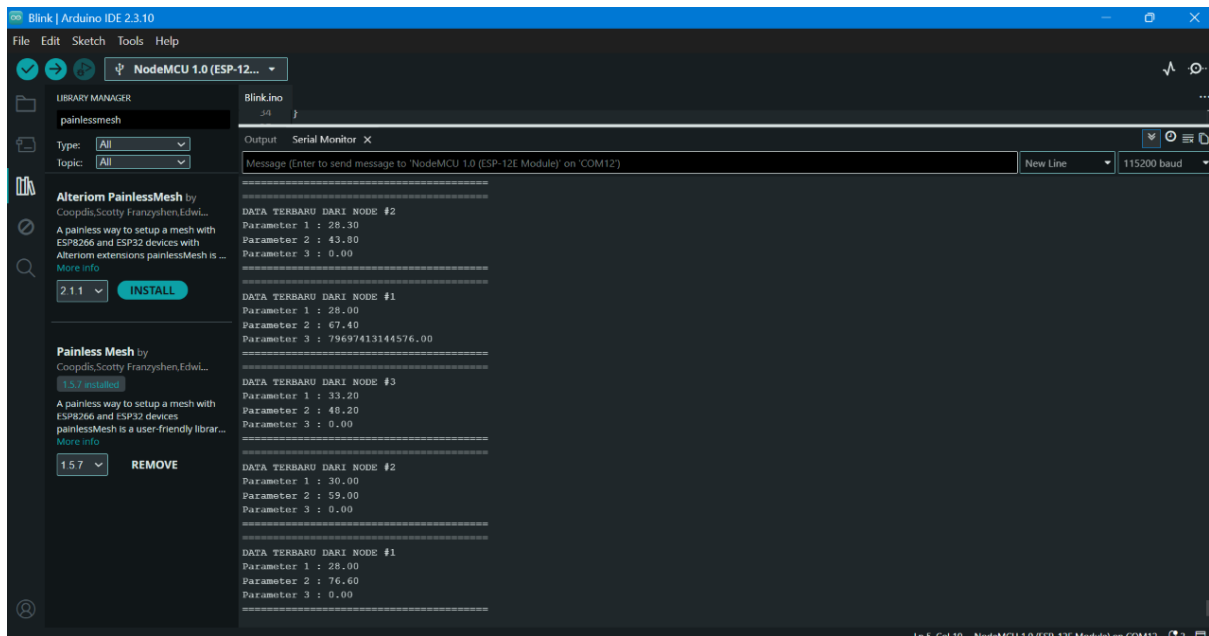
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

Paket masuk dari: bc:dd:c2:89:a9:8c
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 32

Paket masuk dari: bc:dd:c2:89:a9:8c
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 20

5.5.3 Praktikum 3: Pengumpulan Data Telemetry Many-to-One (ESP-NOW)

Controller



The screenshot shows the Arduino IDE interface with a sketch named "Blink.ino" for a NodeMCU 1.0 (ESP-12E) module. The sketch is using the "PainlessMesh" library. The serial monitor shows the output of the sketch, which includes the status of the mesh network and the data received from three nodes.

```
1 #include <PainlessMesh.h>
2 #include <Arduino.h>
3
4 PainlessMesh mesh;
5
6 void setup() {
7   mesh.begin();
8   mesh.setMeshNetwork(1);
9   mesh.setBroadcastKey(1);
10  mesh.setBroadcastKey(1);
11  mesh.setBroadcastKey(1);
12  mesh.setBroadcastKey(1);
13  mesh.setBroadcastKey(1);
14  mesh.setBroadcastKey(1);
15  mesh.setBroadcastKey(1);
16  mesh.setBroadcastKey(1);
17  mesh.setBroadcastKey(1);
18  mesh.setBroadcastKey(1);
19  mesh.setBroadcastKey(1);
20  mesh.setBroadcastKey(1);
21  mesh.setBroadcastKey(1);
22  mesh.setBroadcastKey(1);
23  mesh.setBroadcastKey(1);
24  mesh.setBroadcastKey(1);
25  mesh.setBroadcastKey(1);
26  mesh.setBroadcastKey(1);
27  mesh.setBroadcastKey(1);
28  mesh.setBroadcastKey(1);
29  mesh.setBroadcastKey(1);
30  mesh.setBroadcastKey(1);
31  mesh.setBroadcastKey(1);
32  mesh.setBroadcastKey(1);
33  mesh.setBroadcastKey(1);
34  mesh.setBroadcastKey(1);
35  mesh.setBroadcastKey(1);
36  mesh.setBroadcastKey(1);
37  mesh.setBroadcastKey(1);
38  mesh.setBroadcastKey(1);
39  mesh.setBroadcastKey(1);
40  mesh.setBroadcastKey(1);
41  mesh.setBroadcastKey(1);
42  mesh.setBroadcastKey(1);
43  mesh.setBroadcastKey(1);
44  mesh.setBroadcastKey(1);
45  mesh.setBroadcastKey(1);
46  mesh.setBroadcastKey(1);
47  mesh.setBroadcastKey(1);
48  mesh.setBroadcastKey(1);
49  mesh.setBroadcastKey(1);
50  mesh.setBroadcastKey(1);
51  mesh.setBroadcastKey(1);
52  mesh.setBroadcastKey(1);
53  mesh.setBroadcastKey(1);
54  mesh.setBroadcastKey(1);
55  mesh.setBroadcastKey(1);
56  mesh.setBroadcastKey(1);
57  mesh.setBroadcastKey(1);
58  mesh.setBroadcastKey(1);
59  mesh.setBroadcastKey(1);
60  mesh.setBroadcastKey(1);
61  mesh.setBroadcastKey(1);
62  mesh.setBroadcastKey(1);
63  mesh.setBroadcastKey(1);
64  mesh.setBroadcastKey(1);
65  mesh.setBroadcastKey(1);
66  mesh.setBroadcastKey(1);
67  mesh.setBroadcastKey(1);
68  mesh.setBroadcastKey(1);
69  mesh.setBroadcastKey(1);
70  mesh.setBroadcastKey(1);
71  mesh.setBroadcastKey(1);
72  mesh.setBroadcastKey(1);
73  mesh.setBroadcastKey(1);
74  mesh.setBroadcastKey(1);
75  mesh.setBroadcastKey(1);
76  mesh.setBroadcastKey(1);
77  mesh.setBroadcastKey(1);
78  mesh.setBroadcastKey(1);
79  mesh.setBroadcastKey(1);
80  mesh.setBroadcastKey(1);
81  mesh.setBroadcastKey(1);
82  mesh.setBroadcastKey(1);
83  mesh.setBroadcastKey(1);
84  mesh.setBroadcastKey(1);
85  mesh.setBroadcastKey(1);
86  mesh.setBroadcastKey(1);
87  mesh.setBroadcastKey(1);
88  mesh.setBroadcastKey(1);
89  mesh.setBroadcastKey(1);
90  mesh.setBroadcastKey(1);
91  mesh.setBroadcastKey(1);
92  mesh.setBroadcastKey(1);
93  mesh.setBroadcastKey(1);
94  mesh.setBroadcastKey(1);
95  mesh.setBroadcastKey(1);
96  mesh.setBroadcastKey(1);
97  mesh.setBroadcastKey(1);
98  mesh.setBroadcastKey(1);
99  mesh.setBroadcastKey(1);
100 mesh.setBroadcastKey(1);
```

Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM12')

DATA TERBARU DARI NODE #2
Parameter 1 : 28.30
Parameter 2 : 43.80
Parameter 3 : 0.00

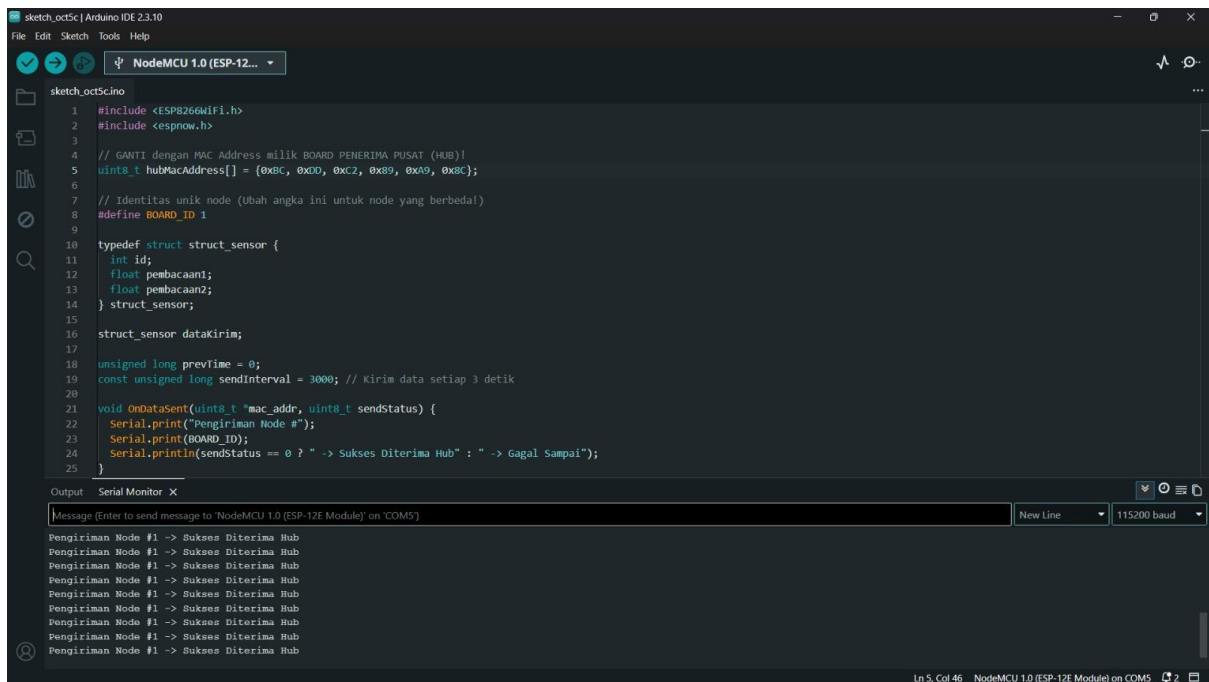
DATA TERBARU DARI NODE #1
Parameter 1 : 28.00
Parameter 2 : 67.40
Parameter 3 : 79697413144576.00

DATA TERBARU DARI NODE #3
Parameter 1 : 33.20
Parameter 2 : 48.20
Parameter 3 : 0.00

DATA TERBARU DARI NODE #2
Parameter 1 : 30.00
Parameter 2 : 59.00
Parameter 3 : 0.00

DATA TERBARU DARI NODE #1
Parameter 1 : 28.00
Parameter 2 : 76.60
Parameter 3 : 0.00

Penerima 1



```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)
5 uint8_t hubMacAddress[] = {0xBC, 0xD0, 0xC2, 0xA9, 0xA9, 0x8C};
6
7 // Identitas unik node (Ubah angka ini untuk node yang berbeda)
8 #define BOARD_ID 1
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
17
18 unsigned long prevTime = 0;
19 const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik
20
21 void onDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
22     Serial.print("Pengiriman Node #");
23     Serial.print(BOARD_ID);
24     Serial.println(sendStatus == 0 ? " -> Sukses Diterima Hub" : " -> Gagal Sampai");
25 }
```

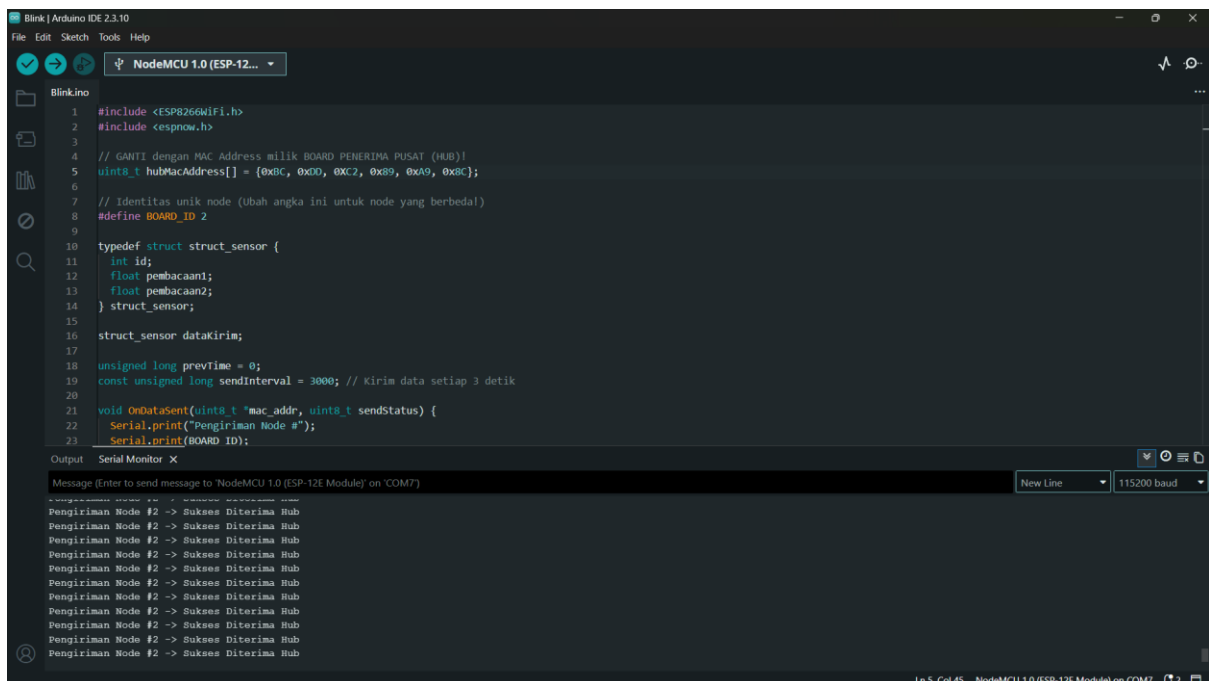
Output Serial Monitor X

Message [Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5']

Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub

Ln 5, Col 46 NodeMCU 1.0 (ESP-12E Module) on COM5

Penerima 2



```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)
5 uint8_t hubMacAddress[] = {0xBC, 0xD0, 0xC2, 0xA9, 0xA9, 0x8C};
6
7 // Identitas unik node (Ubah angka ini untuk node yang berbeda)
8 #define BOARD_ID 2
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
17
18 unsigned long prevTime = 0;
19 const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik
20
21 void onDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
22     Serial.print("Pengiriman Node #");
23     Serial.print(BOARD_ID);
24 }
```

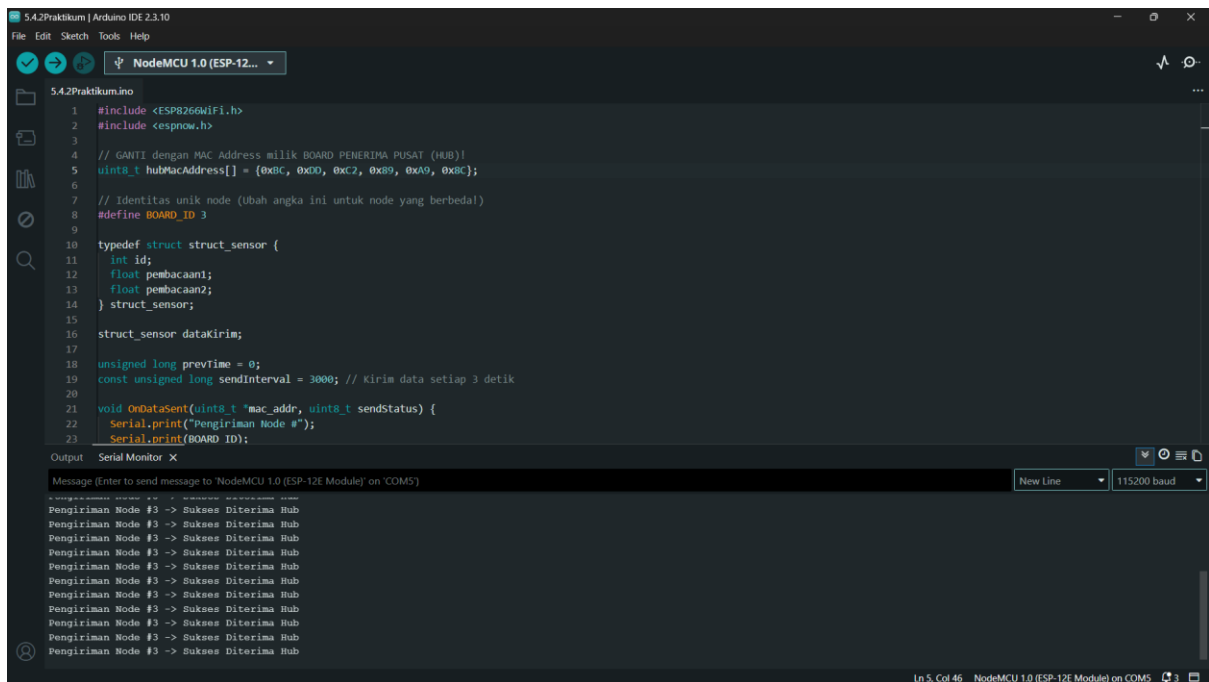
Output Serial Monitor X

Message [Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7']

Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub

Ln 5, Col 45 NodeMCU 1.0 (ESP-12E Module) on COM7

Penerima 3



The screenshot shows the Arduino IDE interface with the following code in the sketch:

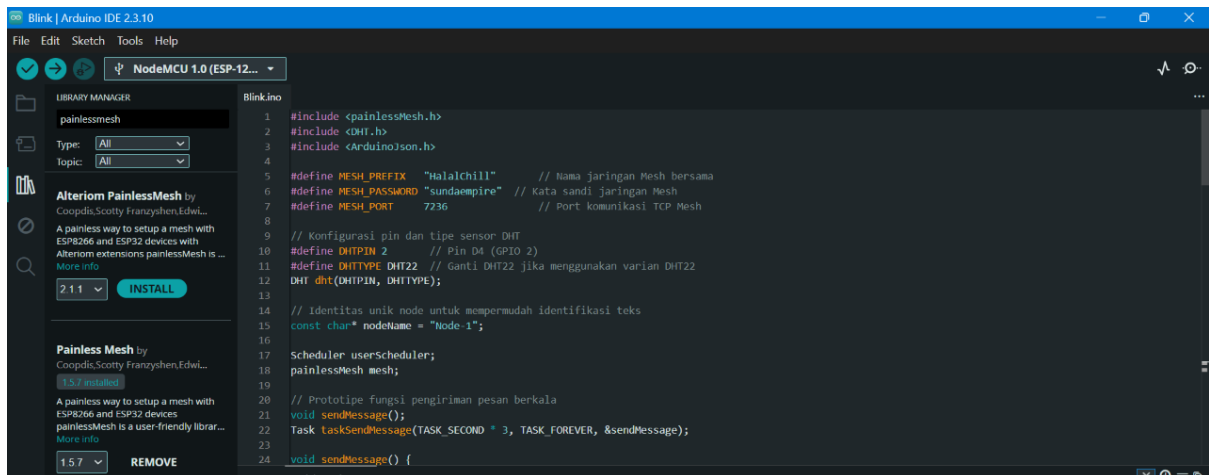
```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)
5 uint8_t hubMacAddress[] = {0xBC, 0x00, 0xC2, 0x09, 0xA9, 0x8C};
6
7 // Identitas unik node (Ubah angka ini untuk node yang berbeda)
8 #define BOARD_ID 3
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
17
18 unsigned long prevTime = 0;
19 const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik
20
21 void onDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
22     Serial.print("Pengiriman Node ");
23     Serial.print(BOARD_ID);
```

The serial monitor shows the following output:

```
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')
Pengiriman Node #3 -> Sukses Diterima Hub
Pengiriman Node #3 -> Sukses Diterima Hub
Pengiriman Node #3 -> Sukses Diterima Hub
Pengiriman Node #3 -> Sukses Diterima Hub
Pengiriman Node #3 -> Sukses Diterima Hub
Pengiriman Node #3 -> Sukses Diterima Hub
Pengiriman Node #3 -> Sukses Diterima Hub
Pengiriman Node #3 -> Sukses Diterima Hub
Pengiriman Node #3 -> Sukses Diterima Hub
Pengiriman Node #3 -> Sukses Diterima Hub
Pengiriman Node #3 -> Sukses Diterima Hub
```

5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

Controller



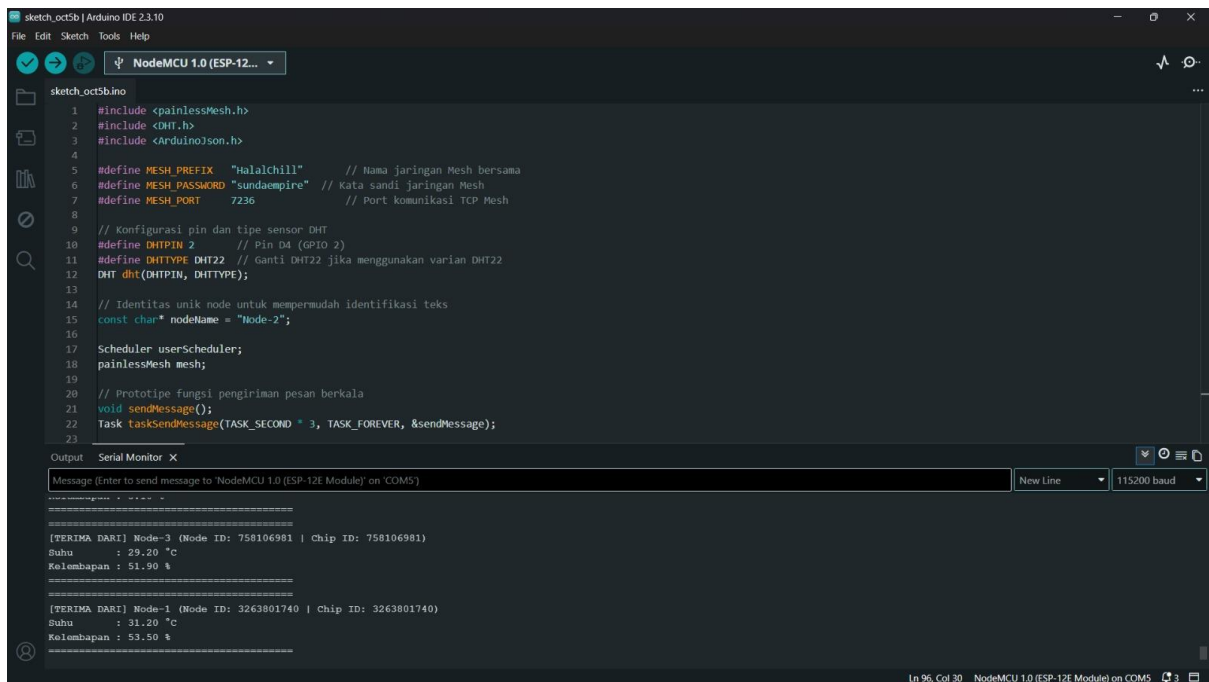
The screenshot shows the Arduino IDE interface with the following code in the sketch:

```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "HalalChilli" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "SundaEmpire" // Kata sandi jaringan Mesh
7 #define MESH_PORT 7236 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-1";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;
19
20 // Prototype fungsi pengiriman pesan berkala
21 void sendMessage();
22 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
23
24 void sendMessage() {
```

The library manager shows the following libraries installed:

- painlessmesh** by Coopdis,Scotty Franyshen,Edwi... (2.1.1) [INSTALL]
- Painless Mesh** by Coopdis,Scotty Franyshen,Edwi... (1.5.7 installed) [REMOVE]

Penerima 1



```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "Halalchill" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "sundaempire" // Kata sandi jaringan Mesh
7 #define MESH_PORT 7236 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-2";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;
19
20 // Prototype fungsi pengiriman pesan berkala
21 void sendMessage();
22 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
23
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

=====

[TERIMA DARI] Node-3 (Node ID: 758106981 | Chip ID: 758106981)
Suhu : 29.20 °C
Kelembapan : 51.80 %

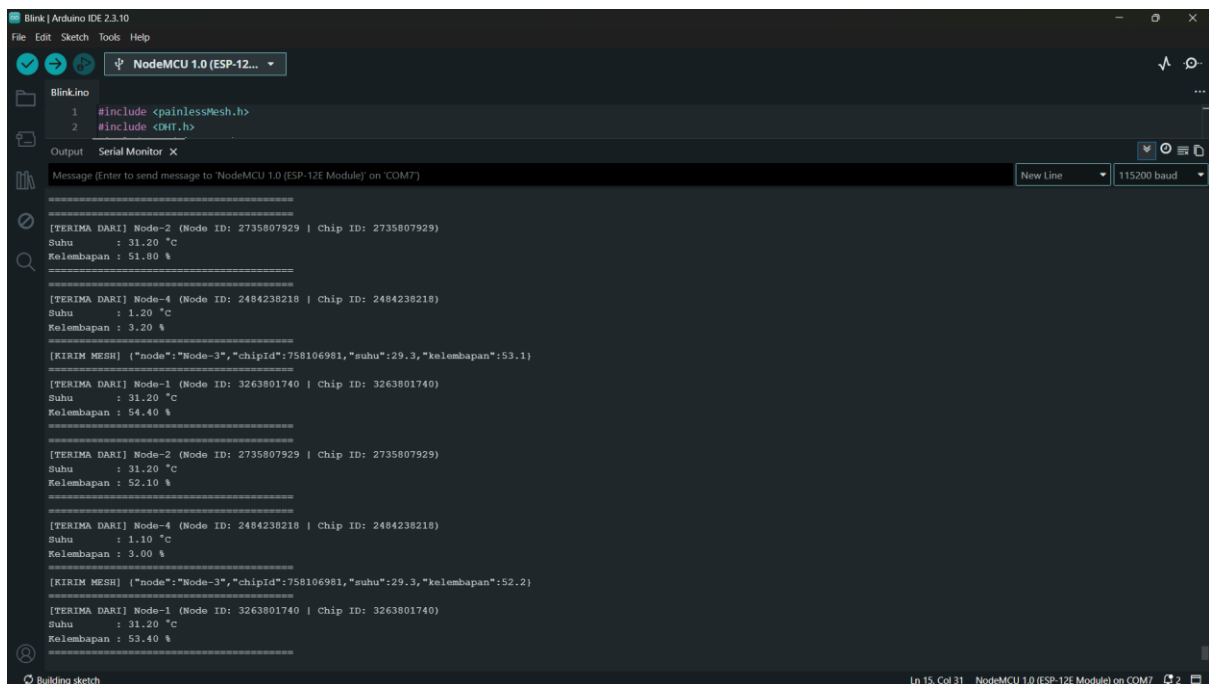
=====

[TERIMA DARI] Node-1 (Node ID: 3263801740 | Chip ID: 3263801740)
Suhu : 31.20 °C
Kelembapan : 53.50 %

=====

Ln 96, Col 30 NodeMCU 1.0 (ESP-12E Module) on COM5

Penerima 2



```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3
4 #define MESH_PREFIX "Halalchill" // Nama jaringan Mesh bersama
5 #define MESH_PASSWORD "sundaempire" // Kata sandi jaringan Mesh
6 #define MESH_PORT 7236 // Port komunikasi TCP Mesh
7
8 // Konfigurasi pin dan tipe sensor DHT
9 #define DHTPIN 2 // Pin D4 (GPIO 2)
10 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
11 DHT dht(DHTPIN, DHTTYPE);
12
13 // Identitas unik node untuk mempermudah identifikasi teks
14 const char* nodeName = "Node-2";
15
16 Scheduler userScheduler;
17 painlessMesh mesh;
18
19 // Prototype fungsi pengiriman pesan berkala
20 void sendMessage();
21 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
22
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')

=====

[TERIMA DARI] Node-2 (Node ID: 2735807929 | Chip ID: 2735807929)
Suhu : 31.20 °C
Kelembapan : 51.80 %

=====

[TERIMA DARI] Node-4 (Node ID: 2484238218 | Chip ID: 2484238218)
Suhu : 1.20 °C
Kelembapan : 3.20 %

=====

[KIRIM MESH] ["node":"Node-3","chipid":"758106981","suhu":29.3,"kelembapan":53.1]

=====

[TERIMA DARI] Node-1 (Node ID: 3263801740 | Chip ID: 3263801740)
Suhu : 31.20 °C
Kelembapan : 54.40 %

=====

[TERIMA DARI] Node-2 (Node ID: 2735807929 | Chip ID: 2735807929)
Suhu : 31.20 °C
Kelembapan : 52.10 %

=====

[TERIMA DARI] Node-4 (Node ID: 2484238218 | Chip ID: 2484238218)
Suhu : 1.10 °C
Kelembapan : 3.00 %

=====

[KIRIM MESH] ["node":"Node-3","chipid":"758106981","suhu":29.3,"kelembapan":52.2]

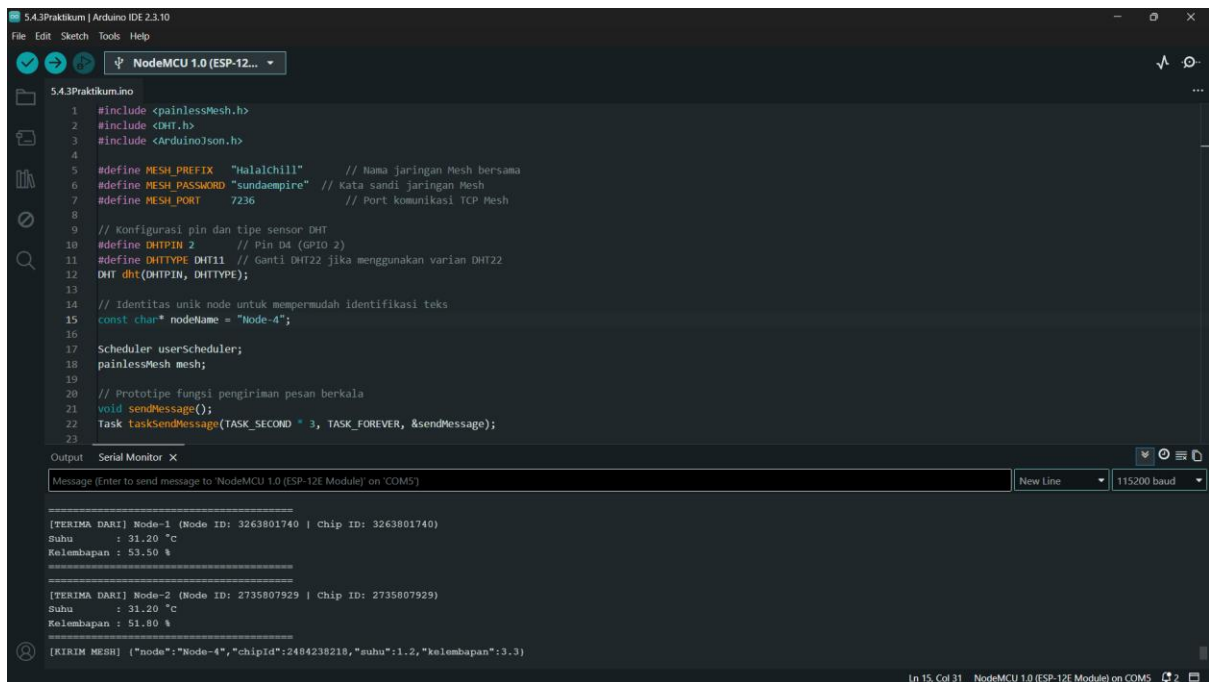
=====

[TERIMA DARI] Node-1 (Node ID: 3263801740 | Chip ID: 3263801740)
Suhu : 31.20 °C
Kelembapan : 53.40 %

=====

Ln 15, Col 31 NodeMCU 1.0 (ESP-12E Module) on COM7

Penerima 3



```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "Halalchill" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "sundaempire" // Kata sandi jaringan Mesh
7 #define MESH_PORT 7236 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT11 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-4";
16
17 Scheduler usersScheduler;
18 painlessMesh mesh;
19
20 // Prototype fungsi pengiriman pesan berkala
21 void sendMessage();
22 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

New Line 115200 baud

```
[TERIMA DARI] Node-1 (Node ID: 3263801740 | Chip ID: 3263801740)
Suhu : 31.20 °C
Kelembapan : 93.50 %
=====
[TERIMA DARI] Node-2 (Node ID: 2735807929 | Chip ID: 2735807929)
Suhu : 31.20 °C
Kelembapan : 91.80 %
=====
[KIRIM MESH] {"node":"Node-4","chipid":2484238218,"suhu":1.2,"kelembapan":3.3}
```

In 15, Col 31 NodeMCU 1.0 (ESP-12E Module) on COM5

5.6 Latihan

Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2): Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (sendStatus == 0)

Jawaban : ESP-NOW mendeteksi kegagalan pengiriman tanpa TCP melalui mekanisme MAC ACK frame di Layer 2 Wi-Fi (IEEE 802.11). Saat data dikirim secara *unicast*, chip radio penerima otomatis membalas dengan sinyal ACK di tingkat *hardware*. Jika penerima mati, sinyal ACK tidak terkirim, sehingga chip pengirim akan melakukan *retry* beberapa kali. Ketika batas percobaan habis tanpa balasan, pengirim menyimpulkan perangkat target tidak aktif dan melaporkan status gagal ke fungsi *callback* OnDataSent.

```

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM12')

TERIMA DARI] Node-3 (Node ID: 758106981 | Chip ID: 758106981)
suhu      : 29.90 °C
kelembapan : 51.00 %

=====
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.2,"kelembapan":54.8}
=====
TERIMA DARI] Node-4 (Node ID: 2484238218 | Chip ID: 2484238218)
suhu      : 1.90 °C
kelembapan : 2.20 %

=====
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.2,"kelembapan":56.1}
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.2,"kelembapan":56.1}
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.3,"kelembapan":56.3}
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.3,"kelembapan":56}
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.2,"kelembapan":55}
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.2,"kelembapan":54.9}
--> Topologi rantai mesh telah diperbarui
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.3,"kelembapan":53.7}
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.3,"kelembapan":53.3}
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.2,"kelembapan":52.4}
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.3,"kelembapan":52.4}
--> Topologi rantai mesh telah diperbarui
--> Koneksi Baru Terdeteksi! Node ID: 2735807929
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.3,"kelembapan":52.6}
=====
TERIMA DARI] Node-2 (Node ID: 2735807929 | Chip ID: 2735807929)
suhu      : 31.30 °C
kelembapan : 51.30 %
=====

```

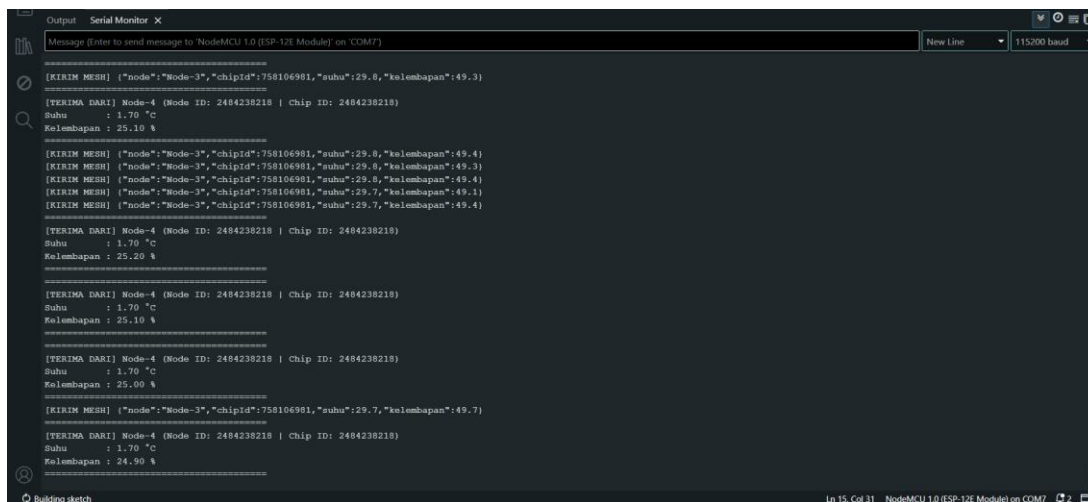
Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4): Jalankan jaringan painlessMesh pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID:

- i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?

Jawaban : 5 Detik

- ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

Jawaban : 3 Detik



```
Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on COM7)
New Line 115200 baud

[KIRIM MESH] {"node":"Node-3","chipId":"758106981","suhu":29.8,"kelembapan":49.3}
[TERIMA DARI] Node-4 (Node ID: 2484238218 | Chip ID: 2484238218)
Suhu : 1.70 °C
Kelembapan : 25.10 %

[KIRIM MESH] {"node":"Node-3","chipId":"758106981","suhu":29.8,"kelembapan":49.4}
[KIRIM MESH] {"node":"Node-3","chipId":"758106981","suhu":29.8,"kelembapan":49.3}
[KIRIM MESH] {"node":"Node-3","chipId":"758106981","suhu":29.8,"kelembapan":49.4}
[KIRIM MESH] {"node":"Node-3","chipId":"758106981","suhu":29.7,"kelembapan":49.1}
[KIRIM MESH] {"node":"Node-3","chipId":"758106981","suhu":29.7,"kelembapan":49.4}

[TERIMA DARI] Node-4 (Node ID: 2484238218 | Chip ID: 2484238218)
Suhu : 1.70 °C
Kelembapan : 25.20 %

[TERIMA DARI] Node-4 (Node ID: 2484238218 | Chip ID: 2484238218)
Suhu : 1.70 °C
Kelembapan : 25.10 %

[TERIMA DARI] Node-4 (Node ID: 2484238218 | Chip ID: 2484238218)
Suhu : 1.70 °C
Kelembapan : 25.00 %

[KIRIM MESH] {"node":"Node-3","chipId":"758106981","suhu":29.7,"kelembapan":49.7}

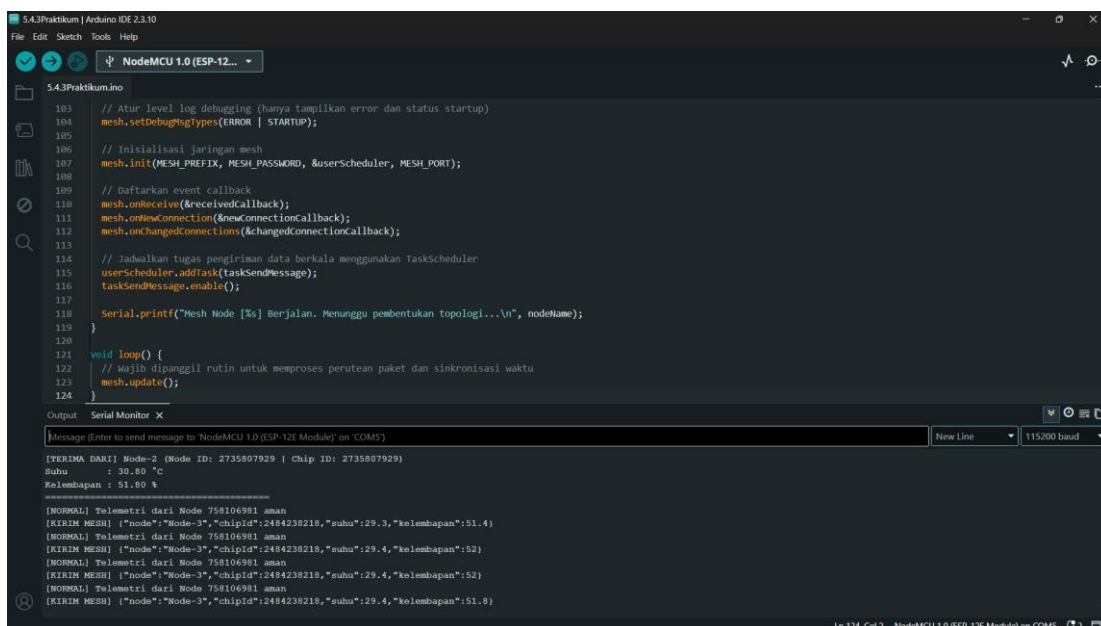
[TERIMA DARI] Node-4 (Node ID: 2484238218 | Chip ID: 2484238218)
Suhu : 1.70 °C
Kelembapan : 24.90 %

Building sketch Ln 15, Col 31 NodeMCU 1.0 (ESP-12E Module) on COM7
```

Latihan 3 (Eksperimen Modifikasi Filter Telemetry & Ambang Batas JSON -

Praktikum 4): Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetry dari semua pengirim tanpa penyaringan.

- Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetry penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
- Jika suhu bernilai normal (≤ 32.0 °C), program cukup mencetak status singkat satu baris: `[NORMAL] Telemetry dari Node <chipId> aman`.
- Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!



```
543Praktikum | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12E)
543Praktikum.ino
103 // Atur level log debugging (hanya tampilkan error dan status startup)
104 mesh.setDebuglogTypes(ERROR | STARTUP);
105
106 // Inisialisasi jaringan mesh
107 mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);
108
109 // Daftarkan event callback
110 mesh.onReceive(&receivedCallback);
111 mesh.onNewConnection(&newConnectionCallback);
112 mesh.onChangedConnections(&changedConnectionCallback);
113
114 // Jadwalkan tugas pengiriman data berkala menggunakan TaskScheduler
115 userScheduler.addTask(taskSendMessage);
116 taskSendMessage.enable();
117
118 Serial.printf("Mesh Node [%s] Berjalan. Menunggu pembentukan topologi...\n", nodeName);
119
120
121 void loop() {
122 // Wajib dipanggil rutin untuk memproses perutean paket dan sinkronisasi waktu
123 mesh.update();
124 }

Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on COM5)
New Line 115200 baud

[TERIMA DARI] Node-2 (Node ID: 2735807929 | Chip ID: 2735807929)
Suhu : 30.80 °C
Kelembapan : 51.80 %

[NORMAL] Telemetry dari Node 758106981 aman
[KIRIM MESH] {"node":"Node-3","chipId":"2484238218","suhu":29.3,"kelembapan":51.4}
[NORMAL] Telemetry dari Node 758106981 aman
[KIRIM MESH] {"node":"Node-3","chipId":"2484238218","suhu":29.4,"kelembapan":52}
[NORMAL] Telemetry dari Node 758106981 aman
[KIRIM MESH] {"node":"Node-3","chipId":"2484238218","suhu":29.4,"kelembapan":52}
[NORMAL] Telemetry dari Node 758106981 aman
[KIRIM MESH] {"node":"Node-3","chipId":"2484238218","suhu":29.4,"kelembapan":51.8}

Ln 124, Col 2 NodeMCU 1.0 (ESP-12E Module) on COM5
```